GPS and Astronomical Commands Reference

1. Overview

The functionality is split into four commands:

- STAR: Calculates the position of a celestial object using the current time and location from the connected GPS module.

- LOCATION: Sets a manual location (latitude/longitude) and time (UTC) for use with the ASTRO command.

- ASTRO: Calculates the position of a celestial object using the manual time and location set by the LOCATION command.

- SLEW: Computes the motor deltas required to point a polar-aligned German Equatorial Mount (GEM) at a target, automatically detecting and resolving meridian flips.

Both STAR and ASTRO share the same syntax for specifying the target object and output variables.

2. Command Syntax

2.1 The STAR Command

Uses live GPS data. Requires a valid GPS fix. The time used is the GPS's UTC time: a time offset given to OPEN ... AS GPS is not applied.

Syntax 1: Named Object

```
STAR object   alt, az [, ra_out, dec_out]
STAR "name"   alt, az [, ra_out, dec_out]
STAR name$    alt, az [, ra_out, dec_out]
```

The object name can be specified in three ways:

- Plain keyword (no quotes, space separator): STAR MOON alt, az

- String literal (quoted, space or comma separator): STAR "Aldebaran", alt, az or STAR "Aldebaran" alt, az

- String variable (space or comma separator): STAR s$, alt, az or STAR s$ alt, az where s is declared DIM s AS STRING

Parameters:

- object / "name" / name$: The celestial body to locate. See Appendix A for the full catalog.

- alt: Variable to store the calculated Altitude (degrees above horizon).

- az: Variable to store the calculated Azimuth (degrees, 0 = North, 90 = East).

- ra_out (Optional): Variable to store the object's current Right Ascension (hours).

- dec_out (Optional): Variable to store the object's current Declination (degrees).

Examples

```
' Plain keyword -- space separator
STAR SATURN alt, az

' String literal -- comma separator
STAR "Saturn", alt, az, ra, dec
```

```
' String variable -- comma separator
DIM target$ = "Saturn"
STAR target$, alt, az

' String variable declared without $
DIM target AS STRING
target = "Saturn"
STAR target, alt, az
```

Syntax 2: Manual Coordinates

```
STAR alt, az, ra, dec [, pm_ra, pm_dec] [, ra_out, dec_out]
```

- alt, az: Variables to store the results.
- ra: Right Ascension of the target in J2000.0 epoch (hours).
- dec: Declination of the target in J2000.0 epoch (degrees).
- pm_ra (Optional): Proper motion in RA (arcseconds/year). Default is 0.
- pm_dec (Optional): Proper motion in Dec (arcseconds/year). Default is 0.
- ra_out, dec_out (Optional): Variables to store the coordinates after precession to the current epoch.

2.2 The LOCATION Command

Sets the context for the ASTRO and SLEW commands.

Syntax

```
LOCATION date$, lat, long [, sidereal_out]
```

- date$: A string containing the date and time in the format "dd/mm/yyyy hh:mm:ss". The separators can be -, /, :, or space. The time is UTC (Universal Time), not local time.
- lat: Latitude in degrees (negative for South).
- long: Longitude in degrees (negative for West).
- sidereal_out (Optional): Variable to store the calculated Local Sidereal Time (hours).

Example

```
LOCATION "25/12/2025 22:30:00", -33.86, 151.21, lst
```

Local time

Subtract your time zone's offset from UTC, including any daylight saving, before LOCATION. Times found from ASTRO's results, such as when an object rises or sets, are then in UTC too, so add the offset back. A local time given as it is makes every result wrong by the offset. For example, in a time zone one hour ahead of UTC:

```
tz = 1                                    ' hours ahead of UTC
' 17:00 local time is 16:00 UTC
t = EPOCH("01-11-2026 17:00:00") - tz * 3600
LOCATION DATETIME$(t), 47.50, 19.04
ASTRO MOON alt, az
```

2.3 The ASTRO Command

Identical to STAR but uses the context set by LOCATION instead of the GPS.

GPS and Astronomical Commands Reference

Syntax

```
ASTRO object     alt, az [, ra_out, dec_out]
ASTRO "name"   , alt, az [, ra_out, dec_out]
ASTRO name$    , alt, az [, ra_out, dec_out]
ASTRO alt, az, ra, dec [, pm_ra, pm_dec] [, ra_out, dec_out]
```

The object name accepts the same three forms as STAR: a plain keyword, a quoted string literal, or a string variable (with or without $ suffix). When a string literal or variable is used a comma is required between the name and the output variables.

2.4 The SLEW Command

Computes the RA and Dec motor deltas needed to slew a polar-aligned GEM from its current pointing position to a target obtained from STAR or ASTRO. Automatically detects whether a meridian flip is required and returns safe flip directions that keep the counterweight down and the tube above the horizon throughout the manoeuvre.

Syntax

```
SLEW dRA1, dDec1, flipRA, flipDec, dRA2, dDec2, mountRA, mountDec, RAs, DECs, LST
```

Output Parameters

- dRA1 (hours, -12..+12): First RA move. Positive = slew east. Always populated.

- dDec1 (degrees): First Dec move. Positive = slew north. Always populated.

- flipRA (hours): RA motor flip movement. +12 = east-to-west flip, -12 = west-to-east flip, 0 = no flip required.

- flipDec (degrees): Dec motor flip movement. +180 = east-to-west flip, -180 = west-to-east flip, 0 = no flip required.

- dRA2 (hours, -12..+12): Second RA move after flip. 0 if no flip required.

- dDec2 (degrees): Second Dec move after flip. Always 0 - Dec is pre-positioned in move 1.

Input Parameters

- mountRA (hours, 0-24): Sky RA the mount is currently pointing at.

- mountDec (degrees): Sky Dec the mount is currently pointing at.

- RAs (hours, 0-24): Target Right Ascension from STAR or ASTRO.

- DECs (degrees): Target Declination from STAR or ASTRO.

- LST (hours, 0-24): Local Sidereal Time from LOCATION.

Meridian Flip Behaviour

A flip is required when the target is on the opposite side of the meridian from the current mount position. This is detected by comparing the signs of the current and target Hour Angles (HA = LST - RA; negative = east of meridian, positive = west).

When a flip is required the caller must execute three steps:

1. Apply dRA1 / dDec1: slews to the meridian and pre-positions Dec to the target value.

2. Apply flipRA / flipDec: the physical flip. The RA motor rotates 12 h (180 deg) to carry the tube to the other side of the pier; the Dec motor rotates 180 deg to re-acquire the target Dec. Directions are chosen to swing the OTA over the top through the polar direction, keeping the counterweight down and the tube above the horizon.

3. Apply dRA2: fine RA correction from the meridian to the target. Dec needs no correction because the flip preserves sky Dec exactly.

When no flip is required, flipRA, flipDec, dRA2, and dDec2 are all 0.

Example

```
LOCATION "07/05/2026 21:30:00", 51.5, -1.8, lst
mount_ra  = lst   ' home: on the meridian
mount_dec = 90.0  ' OTA pointing at celestial pole

ASTRO Saturn alt, az, ra, dec
IF alt < 5 THEN PRINT "Target below horizon" : END

SLEW dra1, ddec1, fra, fdec, dra2, ddec2, mount_ra, mount_dec, ra, dec, lst

drive_RA(dra1) : drive_Dec(ddec1)     ' move 1

IF fra <> 0 THEN
  drive_RA(fra)     ' +12 or -12 hours: swings tube over the top
  drive_Dec(fdec)   ' +180 or -180 degrees: re-acquires Dec
  drive_RA(dra2)    ' fine RA correction to target
END IF

mount_ra = ra : mount_dec = dec
```

Notes

- All RA parameters (dRA1, dRA2, flipRA, mountRA, RAs, LST) are in hours. Multiply by 15 to convert to degrees for stepper motor calculations.

- Dec parameters are in degrees.

- The calling program must track mount_ra and mount_dec, updating them after each successful slew so that subsequent SLEW calls compute correct deltas.

- Obtain LST from LOCATION before calling SLEW; LST changes continuously so it should be refreshed before each slew.

- Check alt > 0 from STAR/ASTRO before calling SLEW to confirm the target is above the horizon.

- Dec motor direction is physically reversed after a meridian flip. The sign of flipDec (+180 or -180) encodes the correct direction; pass it directly to the motor driver without modification.

3. Implementation Details & Math

The implementation performs high-precision astronomical calculations suitable for telescope pointing or navigation.

3.1 Time Systems

- Julian Centuries (T): Time is converted to Julian Centuries from the J2000.0 epoch (2000 Jan 1.5 TT).

- Sidereal Time: Local Sidereal Time (LST) is calculated using a polynomial approximation for Greenwich Mean Sidereal Time (GMST) and the observer's longitude.

GPS and Astronomical Commands Reference

3.2 Planetary Calculations

- Planets: Uses simplified VSOP87-based elements with perturbations for Mercury, Venus, Mars, Jupiter, Saturn, Uranus, and Neptune.

- Moon: Uses a truncated ELP-2000/82 analytical series to calculate the Moon's position, including significant periodic terms for longitude, latitude, and distance.

- Topocentric Correction: For the Moon and planets, the calculation corrects for the observer's position on the Earth's surface (parallax), which is critical for nearby bodies like the Moon.

3.3 Stellar Calculations

- Catalog: Contains J2000.0 coordinates and proper motion data for bright stars and deep sky objects.

- Precession: Coordinates are precessed from the J2000.0 epoch to the current date using the rigorous method described in Meeus (Chapter 21).

- Proper Motion: Applied based on the years elapsed since J2000.0.

3.4 Coordinate Conversion

- Equatorial to Horizontal: Converts Right Ascension and Declination to Altitude and Azimuth using standard spherical trigonometry.

- Refraction: Atmospheric refraction is applied to the Altitude using the standard formula (Meeus Chapter 16), corrected for standard atmospheric pressure and temperature.

GPS and Astronomical Commands Reference

Appendix A: Celestial Catalog

The following objects are recognized by name in the STAR and ASTRO commands.

Solar System Bodies:
SUN, MOON, MERCURY, VENUS, MARS, JUPITER, SATURN, URANUS, NEPTUNE

Stars and Deep Sky Objects:

Achernar	Acrux	Alcyone
Aldebaran	Algenib	Algieba
Algol	Alhajoth	Alhena
Almaak	Alnair	Alnilam
Alnitak	Alphard	Alpheratz
Alpherg	Alrescha	Alsephina
Alshain	Altair	Aludra
Andromeda Galaxy	Antares	Arcturus
Aspidiske	Bellatrix	Betelgeuse
Bodes Galaxy	Canopus	Capella
Caph	Castor	Cigar Galaxy
Deneb	Denebola	Dubhe
Elnath	Eltanin	Enif
Fomalhaut	Gacrux	Hadar
Homam	Kaus Australis	Kochab
Kornephoros	Large Magellanic Cloud	Lesath
Markab	Menkalinan	Mimosa
Mintaka	Mirfak	Nunki
Peacock	Polaris	Pollux
Procyon	Rasalgethi	Rasalhague
Regulus	Rigel	Rigil Kent
Ruchbah	Sabik	Sadalmelik
Sadalsuud	Sadr	Saiph
Scheat	Shaula	Shedir
Sirius	Small Magellanic Cloud	Sombrero Galaxy
Spica	Suhail	Tarazed
Triangulum Galaxy	Vega	Whirlpool Galaxy
Zubenelgenubi	Zubeneschamali	